NOKIA FLAPPY BIRD GAME DEVELOPMENT USING PYTHON

AIM

To design and develop an interactive 2D Nokia Flappy Bird game using Python Tkinter that allows players to control a bird using keyboard keys, navigate through pipes, and increase their score by avoiding obstacles. The game includes a graphical interface, gravity-based bird movement, random pipe generation, score tracking, collision detection, game-over handling, and pause and restart options.

PROCEDURE

1.Import the tkinter library to create the graphical user interface and random to generate random pipe positions.

2.Define the game window dimensions, bird size, pipe width, gap, gravity and movement speed.

3.Create the NokiaFlappyBird class to implement the game functionalities.

4.Initialize the game window and create a canvas with a sky-blue background.

5.Initialize the bird's position, velocity, score, pipes, and game state.

6.Define functions to add pipes at random vertical positions with suitable gaps.

7.Handle keyboard inputs to control the bird, pause the game, and restart it.

8.Implement gravity and bird movement to simulate flying.

9.Move the pipes across the screen and generate new pipes when required.

10.Detect collisions between the bird, pipes, and screen boundaries to end the game.

11.Draw the background, clouds, hills, bird, pipes, score, and game-over screen on the canvas.

12.Run the game loop continuously using root.after() and start the application using root.mainloop()

PROGRAM CODE

```
import tkinter as tk

import random


WIDTH, HEIGHT = 420, 620

BIRD_X = 95

BIRD_R = 14

PIPE_W = 62

PIPE_GAP = 165

GRAVITY = 0.42

FLAP_POWER = -7.4

PIPE_SPEED = 3.0


class NokiaFlappyBird:

    def __init__(self, root):

        self.root = root

        root.title("Nokia Flappy Bird -
Tkinter")

        root.resizable(False, False)

        self.canvas = tk.Canvas(root,
width=WIDTH, height=HEIGHT,

                                bg="#70c5ce",
highlightthickness=0)

        self.canvas.pack()

        self.keys = set()

        root.bind("<KeyPress>",
self.key_down)

        root.bind("<KeyRelease>",
self.key_up)

        self.state = "title"
```

```
        self.best = 0
        self.reset()
        self.state = "title"
        self.root.after(16, self.loop)

    def reset(self):
        self.bird_y = HEIGHT * 0.43
        self.bird_vy = 0.0
        self.pipes = []
        self.score = 0
        self.frame = 0
        self.pipe_timer = 0
        self.ground_offset = 0
        self.add_pipe(WIDTH + 60)
        self.add_pipe(WIDTH + 250)

    def add_pipe(self, x):
        gap_y = random.randint(180,
HEIGHT - 190)
        self.pipes.append({"x": float(x),
"gap_y": gap_y, "scored": False})

    def key_down(self, event):
        key = event.keysym.lower()
        self.keys.add(key)
        if key in ("space", "up",
"return"):
            if self.state in ("title", "dead"):
                self.reset()
                self.state = "playing"
```

```
            self.bird_vy = FLAP_POWER
        elif self.state == "playing":
            self.bird_vy = FLAP_POWER
    elif key == "p":
        if self.state == "playing":
            self.state = "paused"
        elif self.state == "paused":
            self.state = "playing"
    elif key == "r":
        self.reset()
        self.state = "playing"

  def key_up(self, event):

self.keys.discard(event.keysym.lowe
r())

  def update(self):
    self.frame += 1
    self.bird_vy += GRAVITY
    self.bird_y += self.bird_vy
    self.ground_offset =
(self.ground_offset + PIPE_SPEED)
% 24

    for pipe in self.pipes:
        pipe["x"] -= PIPE_SPEED
        if not pipe["scored"] and
pipe["x"] + PIPE_W < BIRD_X:
            pipe["scored"] = True
            self.score += 1
```

```
            self.best = max(self.best,
self.score)



        self.pipes = [p for p in self.pipes
if p["x"] + PIPE_W > -5]

        if not self.pipes or self.pipes[-
1]["x"] < WIDTH - 205:

            self.add_pipe(WIDTH + 10)



        # Screen boundaries

        if self.bird_y - BIRD_R < 0 or
self.bird_y + BIRD_R > HEIGHT - 48:

            self.state = "dead"

            return



        # Pipe collision

        for pipe in self.pipes:

            px = pipe["x"]

            gap_y = pipe["gap_y"]

            if BIRD_X + BIRD_R > px and
BIRD_X - BIRD_R < px + PIPE_W:

                if self.bird_y - BIRD_R <
gap_y - PIPE_GAP/2 or self.bird_y +
BIRD_R > gap_y + PIPE_GAP/2:

                    self.state = "dead"

                    return



    def draw_background(self):

        self.canvas.create_rectangle(0,
0, WIDTH, HEIGHT, fill="#70c5ce",
outline="")

        # clouds
```

```
        for x, y in [(55, 95), (250, 145),
(350, 75)]:

            self.canvas.create_oval(x, y,
x+50, y+18, fill="#d9f1ee",
outline="")

            self.canvas.create_oval(x+15,
y-9, x+42, y+20, fill="#d9f1ee",
outline="")

        # distant hills

        self.canvas.create_oval(-80, 430,
180, 565, fill="#a8d86d",
outline="")

        self.canvas.create_oval(150,
445, 470, 575, fill="#95cc5b",
outline="")

        # ground

        self.canvas.create_rectangle(0,
HEIGHT-48, WIDTH, HEIGHT,
fill="#ded895", outline="")

        self.canvas.create_rectangle(0,
HEIGHT-48, WIDTH, HEIGHT-40,
fill="#74bf45", outline="")

        for x in range(-24, WIDTH+24,
24):

            sx = x - self.ground_offset

            self.canvas.create_line(sx,
HEIGHT-35, sx+12, HEIGHT-35,
fill="#b5a96d", width=2)



    def draw_bird(self):

        y = self.bird_y

        # pixel-art style yellow bird

        self.canvas.create_oval(BIRD_X-
BIRD_R, y-BIRD_R, BIRD_X+BIRD_R,
y+BIRD_R,

                                fill="#ffdf45",
outline="#d99a2b", width=2)
```

```
self.canvas.create_oval(BIRD_X+2, y-
8, BIRD_X+10, y,

                        fill="white",
outline="#a67b25")


self.canvas.create_oval(BIRD_X+6, y-
6, BIRD_X+9, y-2,

                        fill="#1b2633",
outline="")


self.canvas.create_polygon(BIRD_X+
10, y, BIRD_X+22, y+4, BIRD_X+10,
y+7,

                          fill="#f07832",
outline="#c85a24")

      wing_y = y + (4 if self.frame % 8
< 4 else -2)

      self.canvas.create_oval(BIRD_X-
10, wing_y-2, BIRD_X+3, wing_y+7,

                        fill="#f7b92f",
outline="#d99a2b")



   def draw_pipes(self):

      for pipe in self.pipes:

         x = pipe["x"]

         gap_y = pipe["gap_y"]

         top_bottom = gap_y -
PIPE_GAP/2

         bottom_top = gap_y +
PIPE_GAP/2

         # upper pipe

         self.canvas.create_rectangle(x,
0, x+PIPE_W, top_bottom,


fill="#45b84a", outline="#247e32",
width=3)
```

```
self.canvas.create_rectangle(x-5,
top_bottom-18, x+PIPE_W+5,
top_bottom,

                             fill="#55ce58",
outline="#247e32", width=3)

        # lower pipe

        self.canvas.create_rectangle(x,
bottom_top, x+PIPE_W, HEIGHT-48,


fill="#45b84a", outline="#247e32",
width=3)


self.canvas.create_rectangle(x-5,
bottom_top, x+PIPE_W+5,
bottom_top+18,

                             fill="#55ce58",
outline="#247e32", width=3)



    def overlay(self, title, subtitle):

        self.canvas.create_rectangle(35,
225, WIDTH-35, 370,

                           fill="#ffffff",
outline="#3d7d87", width=3)


self.canvas.create_text(WIDTH//2,
270, text=title,

                        fill="#24434a",
font=("Arial", 22, "bold"))


self.canvas.create_text(WIDTH//2,
315, text=subtitle,

                        fill="#355c64",
font=("Arial", 10))



    def draw(self):

        self.canvas.delete("all")

        self.draw_background()
```

```
        self.draw_pipes()

        self.draw_bird()

        self.canvas.create_rectangle(8,
8, 150, 45, fill="#ffffff",
outline="#438a91")

        self.canvas.create_text(18, 17,
anchor="w", text=f"SCORE:
{self.score}",

                                fill="#24434a",
font=("Arial", 11, "bold"))

        self.canvas.create_text(18, 34,
anchor="w", text=f"BEST:
{self.best}",

                                fill="#24434a",
font=("Arial", 9, "bold"))

        if self.state == "paused":

            self.overlay("PAUSED", "Press
P to continue")

        elif self.state == "dead":

            self.overlay("GAME OVER",
f"Score: {self.score}  |  Press SPACE
to retry")



    def title_screen(self):

        self.canvas.delete("all")

        self.draw_background()

        self.draw_pipes()

        self.bird_y = HEIGHT * 0.43 +
(random.randint(-2, 2) if self.frame
% 12 == 0 else 0)

        self.draw_bird()


self.canvas.create_text(WIDTH//2,
120, text="NOKIA FLAPPY BIRD",

                                fill="#ffffff",
font=("Arial", 25, "bold"))
```

```
self.canvas.create_text(WIDTH//2,
165, text="TAP TO FLY • AVOID
THE PIPES",

                        fill="#24434a",
font=("Arial", 11, "bold"))


self.canvas.create_text(WIDTH//2,
420, text="PRESS SPACE OR ENTER
TO START",

                        fill="#ffffff",
font=("Arial", 13, "bold"))


self.canvas.create_text(WIDTH//2,
455, text="SPACE / UP = FLAP     P =
PAUSE     R = RESTART",

                        fill="#24434a",
font=("Arial", 9, "bold"))



    def loop(self):

        if self.state == "title":

            self.frame += 1

            self.title_screen()

        elif self.state == "playing":

            self.update()

            self.draw()

        else:

            self.draw()

        self.root.after(16, self.loop)



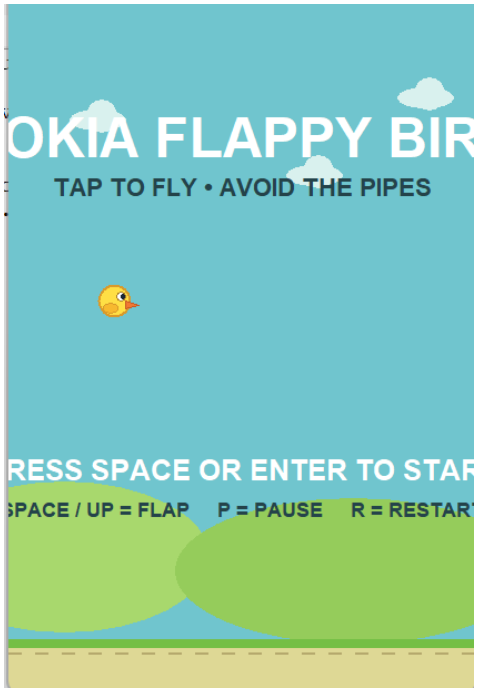
if __name__ == "__main__":

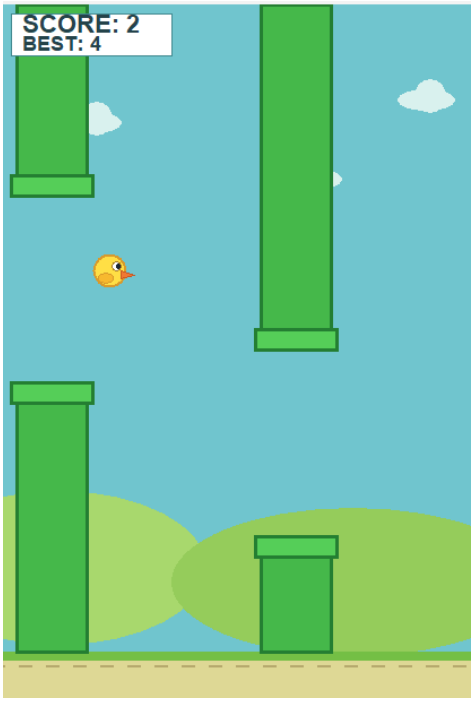
    root = tk.Tk()

    game = NokiaFlappyBird(root)

    root.mainloop()
```

output:





RESULT:

Thus, an interactive 2D Nokia Flappy Bird game was developed using Python Tkinter with graphical game design, keyboard-based bird movement, gravity simulation, randomly generated pipes, automatic score calculation, collision detection, game-over handling, pause and restart functionality. The program demonstrates event-driven programming, GUI design, animation, random number generation, conditional statements, and basic game development concepts.